



ARCOS

Sistemas Distribuidos
Grado en Ingeniería Informática

Ejercicio Evaluable 1: Colas de mensajes

Miembros del grupo:

Alejandro Quirante Sanz - 100522183@alumnos.uc3m.es
Álvaro de la Roza Garcia - 100495871@alumnos.uc3m.es

Grupo XX

2025/2026

1. Introducción

El objetivo de este proyecto consiste en la implementación de un sistema de almacenamiento de tuplas con formato `<key, value1, value2, value3>`. El desarrollo se lleva a cabo a través de dos versiones: la primera monolítica y la segunda distribuida, ambas implementaciones usan librerías dinámicas.

Para la comunicación entre procesos, se han utilizado exclusivamente colas de mensajes POSIX, y para la gestión del almacenamiento se ha optado por una estructura de datos dinámica. Este documento describe el diseño técnico, la implementación del protocolo de comunicación, el manejo de errores y las pruebas realizadas para asegurar el cumplimiento de los requisitos de atomicidad y concurrencia.

2. Diseño de la Versión No Distribuida (Apartado A)

2.1. Gestión de Almacenamiento Dinámico

Siguiendo las especificaciones del enunciado, el servicio no debe imponer un límite en el número de elementos almacenados. Por ello, en el archivo `claves.c` se ha implementado una **Lista Enlazada Simple**.

Cada elemento se almacena en un nodo que contiene la clave (cadena de hasta 255 caracteres), el valor 1 (cadena), el valor 2 (vector de floats de tamaño N entre 1 y 32) y el valor 3 (estructura Paquete). El uso de `malloc` y `free` permite que la capacidad del sistema no dependa del stack y que se pueda hacer una gestión de memoria más eficiente y útil, controlandola en cada momento.

2.2. Atomicidad y Concurrencia

Para garantizar que las operaciones sobre la lista sean atómicas, hemos implementado un mutex global de forma que cuando se inicia una operación sobre el recurso (lista enlazada) no se puede conducir otra que interfiera. Hemos optado por proteger de esta forma todas las funciones ya que en este sistema hasta las lecturas o búsquedas pueden ser críticas si se llevan a cabo durante otra operación.

Por ejemplo: Si insertamos un nodo y el planificador del sistema operativo decide cambiar el proceso en ejecución a uno que busque este nodo, dará error porque no se terminó de insertar.

2.3. Generación de la Biblioteca Dinámica

La lógica de `claves.c` se compila como una biblioteca dinámica denominada `libclaves.so`. Esto permite que cualquier aplicación cliente pueda hacer uso de los servicios de almacenamiento simplemente enlazando la biblioteca en tiempo de ejecución, separando claramente la API de la interfaz

3. Diseño de la Versión Distribuida (Apartado B)

La versión distribuida separa físicamente la lógica de almacenamiento del cliente mediante una arquitectura local basada en colas de mensajes.

3.1. Servidor

El servidor es el núcleo del sistema distribuido. Sus principales responsabilidades son:

- **Gestión de Peticiones:** Crea una cola de mensajes conocida (`/servidor_mq`) con permisos de solo lectura para recibir mensajes de los clientes.
- **Concurrencia mediante Hilos:** Por cada mensaje recibido, el servidor lanza un hilo de ejecución independiente (`pthread_create`). Este hilo procesa la petición utilizando la biblioteca `libclaves.so` y envía el resultado a la cola de respuesta específica del cliente.
- **Limpieza:** Implementa un manejador de señales para `SIGINT` y `SIGTERM`, asegurando que las colas se eliminen correctamente del sistema de archivos `/dev/mqueue` al finalizar la ejecución. De igual forma se asegura de que en cualquier caso los hilos terminen de manera sofisticada y que las colas de mensaje se borren antes de empezar (por si quedase ruido proveniente de alguna ejecución previa u otro proceso) y al finalizar.

3.2. Proxy

El proxy actúa como un intermediario transparente. Implementa la misma API definida en `claves.h`, por lo que el código del cliente no requiere modificaciones.

1. **Identificación Única:** El proxy utiliza el PID del proceso cliente (`getpid()`) para crear una cola de respuesta única: `/cliente_mq_PID`. Esto garantiza que el servidor pueda responder de forma individual a múltiples clientes simultáneos.
2. **Marshaling:** Los argumentos de las funciones se empaquetan en una estructura `RequestMessage`.
3. **Control de Comunicaciones:** Se utiliza `mq_timedreceive` con un tiempo de espera de 5 segundos. Si el servidor no responde, el proxy asume un fallo de red y devuelve un código de error de comunicaciones.

3.3. Protocolo de Mensajería

Se han diseñado dos estructuras de datos fijas para el intercambio de información:

- **RequestMessage:** Incluye el tipo de operación, el ID del cliente y todos los datos de la tupla.
- **ResponseMessage:** Contiene el código de resultado y los campos necesarios para devolver datos en las operaciones de consulta (`get_value` y `exist`).

Nota: El envío de mensajes se realiza con un struct, lo cual sería incorrecto en otros casos, pero que ambos programas estén programados en c y compilados usando el mismo compilador nos asegura que la representación en memoria será igual, lo que nos permite enviar y recibir directamente un struct sin preprocesado.

4. Manejo de Errores

El sistema está diseñado para discriminar entre errores lógicos del servicio y errores críticos de la infraestructura de comunicación.

4.1. Errores de Servicio (-1)

Se devuelve el valor -1 cuando la operación no puede completarse por motivos lógicos:

- **Claves Duplicadas:** Al intentar insertar una clave que ya existe.
- **Claves Inexistentes:** Al intentar obtener o modificar una clave que no está en la lista.
- **Validación de Rangos:** Si el valor `N_value2` no está comprendido entre 1 y 32. Según el enunciado, esta validación debe realizarse en el lado del cliente (proxy) para evitar comunicaciones innecesarias.

4.2. Errores de Comunicación (-2)

Se devuelve -2 cuando se produce un fallo en el sistema de mensajería:

- El servidor no está activo (la cola `/servidor_mq` no existe).
- Error al crear o abrir la cola de respuesta del cliente.
- El mensaje no puede ser enviado por saturación de la cola.
- Se produce un **timeout** esperando la respuesta del servidor.

4.3. Integridad de los Datos

Para evitar desbordamientos de memoria (buffer overflow), el proxy y el servidor validan que las cadenas de caracteres no superen los 255 caracteres. Todas las copias de memoria se realizan utilizando `strncpy` y se fuerza la inserción del carácter nulo `\0` al final de cada buffer antes de procesar la información.

5. Compilación y Ejecución

La gestión de la compilación se realiza mediante **CMake**, que organiza la salida de binarios en directorios de construcción separados.

5.0.1. Construcción del Proyecto

Desde la raíz del proyecto, ejecute los siguientes comandos en la terminal:

```
mkdir build
cd build
cmake -B build
make
```

De esta forma, los archivos de construcción del proyecto se quedan en `./build`, lo que nos facilita borrar los archivos para hacer una construcción limpia. (se adjunta también un archivo `build.sh` que automatiza el proceso)

5.0.2. Ejecución de la Versión No Distribuida

Para ejecutar la versión no distribuida simplemente tenemos que iniciar el cliente que se ubica en: `./Versión no distribuida/build/app_cliente`

5.0.3. Ejecución de la Versión Distribuida

Para ejecutar la versión distribuida debemos iniciar tanto el servidor como el cliente para que puedan comunicarse.

Servidor: Tenemos que ejecutar `./Versión distribuida/build/servidor_mq`

Cliente: En otra terminal, hay que ejecutar `./Versión distribuida/build/app_cliente_dist`

Limpieza: Con tan solo hacer `rm -rf ./build` borramos los archivos de construcción, de la limpieza de las colas no necesitamos ocuparnos ya que el propio programa lo maneja.

6. Pruebas

El código en `app_cliente` y `app_cliente_dist` (que tienen el mismo código, solo que se llaman distinto por como lo especificamos en el archivo `CMakeLists.txt`) ejecuta pruebas para verificar que todo funciona correctamente:

6.1. Pruebas Funcionales

Inserción y Recuperación: Se inserta una tupla con valores heterogéneos y se verifica que `get_value` recupera exactamente los mismos bytes (comparación de strings, vectores de floats y enteros del paquete).

Existencia: Se valida que la función `exist` distingue correctamente entre claves presentes y ausentes.

Modificación: Se comprueba que `modify_value` actualiza todos los campos de una clave sin alterar el resto de la lista.

Borrado: Se elimina una clave y se confirma que las llamadas posteriores a `get_value` devuelven `-1`.

6.2. Pruebas de Capacidad

Carga Masiva: Se realiza un bucle de 100 inserciones consecutivas. El objetivo es comprobar que la lista enlazada gestiona la memoria dinámica sin corromperse y que el servidor concurrente procesa los mensajes sin pérdida de información.

Destrucción Total: Se invoca `destroy()` y se verifica que el sistema vuelve al estado inicial (lista vacía).

6.3. Pruebas de Red y Concurrency

Multi-cliente: Se ejecutan varios clientes simultáneos enviando peticiones al servidor. Se verifica que gracias al ID basado en el PID, el servidor responde a cada cliente en su cola correspondiente.

Fallo del Servidor: Se ejecuta el cliente con el servidor apagado para asegurar que se captura el error y se retorna `-2`.

7. Conclusiones

Se ha logrado una transición de una aplicación monolítica a una distribuida. El uso de listas enlazadas garantiza que no existan límites artificiales en el número de elementos, mientras que el protocolo de colas POSIX y el servidor multihilo proporcionan comunicación para un sistema concurrente y robusto. Como único punto posible a considerar, se podría haber implementado con ficheros en vez de con listas enlazadas, lo que nos blindaría algo más frente a errores inesperados, aunque realentizaría la aplicación al tener que realizar llamadas al sistema para crear estos ficheros, mientras que con la implementación actual, la operación con nodos resulta muy eficiente en este aspecto.